# SEQUENCE-TO-SEQUENCE PIANO TRANSCRIPTION WITH TRANSFORMERS

**Curtis Hawthorne** **Ian Simon** **Rigel Swavely**
**Ethan Manilow*** **Jesse Engel**
Google Research
{fjord,iansimon,rigeljs,emanilow,jesseengel}@google.com

## ABSTRACT

Automatic Music Transcription has seen significant progress in recent years by training custom deep neural networks on large datasets. However, these models have required extensive domain-specific design of network architectures, input/output representations, and complex decoding schemes. In this work, we show that equivalent performance can be achieved using a generic encoder-decoder Transformer with standard decoding methods. We demonstrate that the model can learn to translate spectrogram inputs directly to MIDI-like output events for several transcription tasks. This sequence-to-sequence approach simplifies transcription by jointly modeling audio features and language-like output dependencies, thus removing the need for task-specific architectures. These results point toward possibilities for creating new Music Information Retrieval models by focusing on dataset creation and labeling rather than custom model design.

## 1. INTRODUCTION

Automatic Music Transcription (AMT) is one of the core tasks of Music Information Retrieval (MIR). The objective of AMT is to convert raw audio to a appropriate symbolic representation. In this paper we consider the problem of transcribing piano audio to a series of note events indicating precise onset/offset timings and velocities, as opposed to a sheet music score that is aligned to a metrical grid.

Recent progress in piano transcription has been largely driven by two factors: the construction and release of datasets containing aligned piano audio and MIDI (most notably MAPS [1] and MAESTRO [2]) and the use of deep neural networks with architectures specifically designed for piano transcription (e.g., the Onsets and Frames architecture that models note onsets and note presence separately [3]). While domain-specific models have lead to improvements on benchmark datasets, it is not clear if these approaches can translate to other domains and MIR tasks.

Simultaneously, Transformer models employing self-attention [4] have demonstrated a surprising ability to achieve state-of-the-art results in a variety of domains with the same core architecture, by simply varying the input and output representations [5–14].

In this paper, we demonstrate that a generic Transformer model can achieve state-of-the-art piano transcription without any domain-specific adaptations. Using "off-the-shelf" components (an essentially unmodified encoder-decoder configuration from the T5 paper [6]) and a simple greedy decoding strategy, we train a model to encode raw spectrogram frames and decode directly to a sequence of note events inspired by messages in the original MIDI protocol [15] (e.g., `note on` and `velocity` messages). As such, we refer to the model's output as "MIDI-like" throughout the rest of this paper. We provide details of our model's vocabulary in Section 3.2.

Further, we demonstrate that this domain-agnostic approach enables us to train several variations on the transcription task (e.g., transcribing only note onsets) by changing only the training labels and without modifying the inputs or model.

In summary, this work illustrates the value of using generic sequence-to-sequence Transformers for piano transcription, without domain-specific adaptations, and points to the potential to expand a similar approach to a variety of MIR tasks.

## 2. RELATED WORK

### 2.1 Piano Transcription

Much progress has been made in piano transcription using deep neural network models trained on datasets of aligned audio and MIDI. In 2012, Boulanger-Lewandowski et al. [16] (building off the acoustic model of Nam et al. [17]) trained a recurrent neural network (RNN) transcription model to output a binary piano roll. Böck and Schedl [18] trained a similar RNN-based model for piano onsets only. Hawthorne et al. [3] improved transcription accuracy by having separate convolution-based model *stacks* for detecting note onsets, note presence, and note velocities. Model outputs were decoded into discrete notes using a hard prior by not initiating a note unless the onset predictor gave probability more than 0.5.

* Work done as a Google Brain Student Researcher.



© C. Hawthorne, I. Simon, R. Swavely, E. Manilow, and J. Engel. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** C. Hawthorne, I. Simon, R. Swavely, E. Manilow, and J. Engel, "Sequence-to-Sequence Piano Transcription with Transformers", in *Proc. of the 22nd Int. Society for Music Information Retrieval Conf.,* Online, 2021.

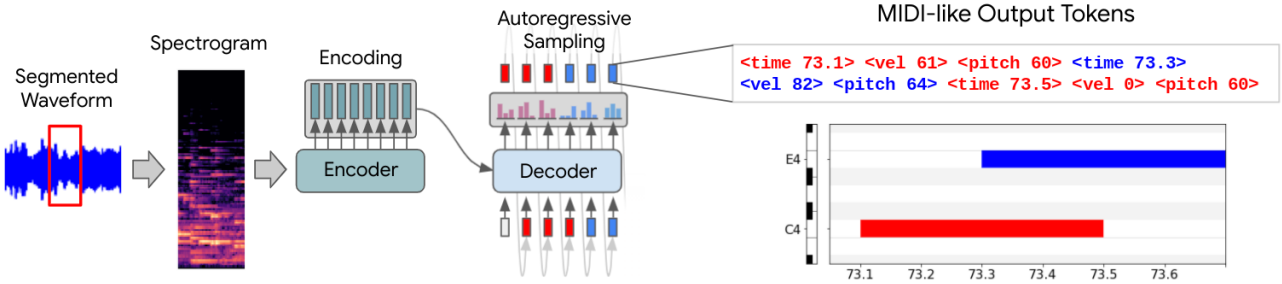



**Figure 1**. Our model is a generic encoder-decoder Transformer architecture where each input position contains a single spectrogram frame and each output position contains an event from our MIDI-like vocabulary. Outputs tokens are autoregressively sampled from the decoder, at each step taking the token with maximum probability.

More recently, progress on piano transcription has largely involved adding more domain-specific deep neural network components and modifying the decoding process. For the most part, this additional complexity has been geared toward the specific purpose of improving piano transcription accuracy.

Kong et al. [19] achieve higher transcription accuracy by using regression to predict precise continuous onset/offset times, using a similar network architecture to Hawthorne et al. [3]. Kim & Bello [20] use an adversarial loss on the transcription output to encourage a transcription model to output more plausible piano rolls. Our sequence-to-sequence approach explicitly models such inter-output dependencies through the autoregressive decoder, which is trained end-to-end with the encoder that extracts meaningful audio features.

Kwon et al. [21] use a language model of sorts to model per-pitch note state transitions instead of having separate onset, frame, and offset stacks. However, the decoding process is fairly complex, in particular the handling of interactions between different pitches. Similarly, Kelz et al. [22] decode using a hidden Markov model over note states based on attack-decay-sustain-release (ADSR) envelopes.

In a very thorough domain-specific treatment, Elowsson [23] constructs a hierarchical model that extracts fundamental frequency contours from spectrograms and uses these contours to infer note onsets and offsets. While it can be useful for many applications to have such intermediate representations as in Engel et al. [24], in this work we treat polyphonic transcription from audio to discrete notes as an end-to-end problem. This has the advantage of conceptual simplicity and our evaluation (Section 4.2) shows it is also effective.

### 2.2 Transformers

Recently, a generic Transformer architecture [4] has been used across multiple domains to solve sequence-to-sequence problems, replacing task-specific architectures that had previously been in use. Outside the field of natural language processing in which Transformers initially emerged and are now widely used (e.g., GPT-3 by Brown et al. [25] and T5 by Raffel et al. [6]), Transformers have been used in computer vision for tasks such as object detection [8], caption-based image generation [9], and pose reconstruction [10], as well as audio-related tasks including speech recognition [11, 12], speech synthesis [13], and audio event classification [14].

Note that many of the above uses of Transformer take advantage of a *pretraining* phase where the model is trained on a large amount of unlabeled data using self-supervision. While it is possible that such a pretraining phase might also help with music transcription, in this work we explore the simpler setting of training the Transformer architecture from scratch on labeled transcriptions in an ordinary supervised fashion.

### 2.3 Sequence-to-Sequence Transcription

The idea of using Transformers for music transcription has also been considered. Awiszus in 2019 [26] explored several formulations of music transcription as a sequence-to-sequence problem, using a variety of input and output representations (including ones similar to our own) with both LSTM [27] and Transformer models. However, the paper was unable to demonstrate clear success, seemingly due to using a framewise multi-F0 evaluation rather than the note-based evaluation standard in piano transcription, using relative time shifts rather than absolute (see Section 3.2), and training on the MAPS dataset which is much smaller than the MAESTRO dataset we use. Earlier, Ullrich and van der Wel [28] appear to be the first to have posed music transcription as a sequence-to-sequence problem (using LSTMs instead of Transformers), but their system could handle only monophonic music.

## 3. MODEL

As mentioned above, our model is a generic encoder-decoder Transformer architecture where each input position contains a single spectrogram frame and each output position contains an event from our MIDI-like vocabulary. An overview of our model and our input and output setup is shown in Figure 1.

Inputs are processed through a stack of encoder self-attention layers resulting in a sequence of embeddings the same length as the original input. A stack of decoder lay-

ers then uses both causally masked self-attention over the decoder output and cross-attention over the full output of the encoder stack. Crucially, this allows the symbolic token output to be variable length, dependent only on the number of tokens needed to describe the input audio.

### 3.1 Model Architecture

The model configuration is based on the "small" model from T5 [6], with modifications as suggested by the T5.1.1 recipe [1]. Specifically, our model uses an embedding size of $d_{\text{model}} = 512$, a feed-forward output dimensionality of $d_{\text{ff}} = 1{,}024$, a key/value dimensionality of $d_{\text{kv}} = 64$, 6-headed attention, and 8 layers each in the encoder and decoder.

Our model has a few minor changes from the standard configuration. Most important is that in order to use continuous spectrogram inputs, we add a dense layer to project each spectrogram input frame to the Transformer's input embedding space. We also use fixed absolute positional embeddings rather than the logarithmically scaled relative positional bucket embeddings used in T5, to ensure all positions can be attended to with equal resolution. Finally, we use `float32` activations for better training stability because our model is small enough that we do not need the memory efficiency of the less precise `bfloat16` format [29] typically used in large T5 models.

The model is implemented using the T5X framework [2], which is built on Flax [30] and JAX [31]. We also use SeqIO [3] for data preprocessing and evaluation. Code for our implementation will be available at `https://goo.gl/magenta/seq2seq-piano-transcription-code`.

While some recent research using Transformers has favored very large models, such as GPT-3 [25] with 175B parameters, we found that a comparatively small model is sufficient for these tasks. With the configuration described above, our model has only 54M parameters, only roughly twice that of Onsets and Frames [2], which has 28M parameters.

### 3.2 Inputs and Outputs

The model uses spectrogram frames as input, with one frame per input position. To match the T5 setup, we terminate input sequences with a learnable EOS (End of Sequence) embedding. The model output at each step is a softmax distribution over a discrete vocabulary of events, described below. This vocabulary is heavily inspired by the messages originally defined in the MIDI specification [15]. Using events as the output representation instead of a piano roll matrix has the advantage of being much more sparse because outputs are needed only when an event occurs, instead of needing to annotate every frame. The vocabulary consists of the following token types:

**Note** [128 values] Indicates a note-on or note-off event for one of the 128 MIDI pitches. We use the full MIDI pitch range for flexibility, but for these experiments, only the 88 pitches corresponding to piano keys are actually used.

**Velocity** [128 values] Indicates a velocity change to be applied to all subsequent Note events (until the next Velocity event). There are 128 velocity values including zero, a special value which causes subsequent Note events to be interpreted as note-off events.

**Time** [6,000 values] Indicates the absolute time location within the segment, quantized into 10 ms bins. This time will apply to all subsequent Note events until the next Time event. Time events must occur in chronological order. We define the vocabulary with times up to 60 seconds for flexibility, but because time resets for each segment, in practice we use only the first few hundred events of this type.

**EOS** [1 value] Indicates the end of the sequence.

Previous work that used such a MIDI-like event vocabulary [32] used relative time shifts between events, indicating the amount of time elapsed since the last time shift. However, in the sequence-to-sequence scenario a single relative time shift error early in the output causes all subsequent output steps to be incorrect, and such errors accumulate as sequence length increases. To adjust for this drift, the Transformer model would have to learn to perform a cumulative sum over all previous time shifts in order to determine the current position in time. We instead use absolute time, where each time event indicates the amount of time from the beginning of the segment, as illustrated in Figure 1. This gives the model the easier task of determining each timestamp independently; we also examine this choice empirically in Section 4.4 and find that using absolute time shifts instead of relative shifts results in much better performance.

We use a temporal resolution of 10 ms for our time events as some experiments have found this displacement to be approximately the limit of human perception [33] (though others have reported smaller values e.g. 5 ms in Handel [34]). We leave open the possibility that our results could be improved further with finer event resolution, for example by predicting continuous times as in Kong et al. [19].

Decoding model output during inference is done with a simple greedy autoregressive algorithm. We choose the maximum probability event at each step and feed that back into the network as the predicted event for that step. We continue this process until the model predicts an EOS token.

Using an event sequence as our training target instead of piano roll matrices or other frame-based formats enables significant flexibility. For example, we demonstrate in Section 4.4 that the exact same model configuration with

[1] `https://github.com/google-research/text-to-text-transfer-transformer/blob/master/released_checkpoints.md#t511`

[2] `https://goo.gle/t5x`

[3] `http://github.com/google/seqio`

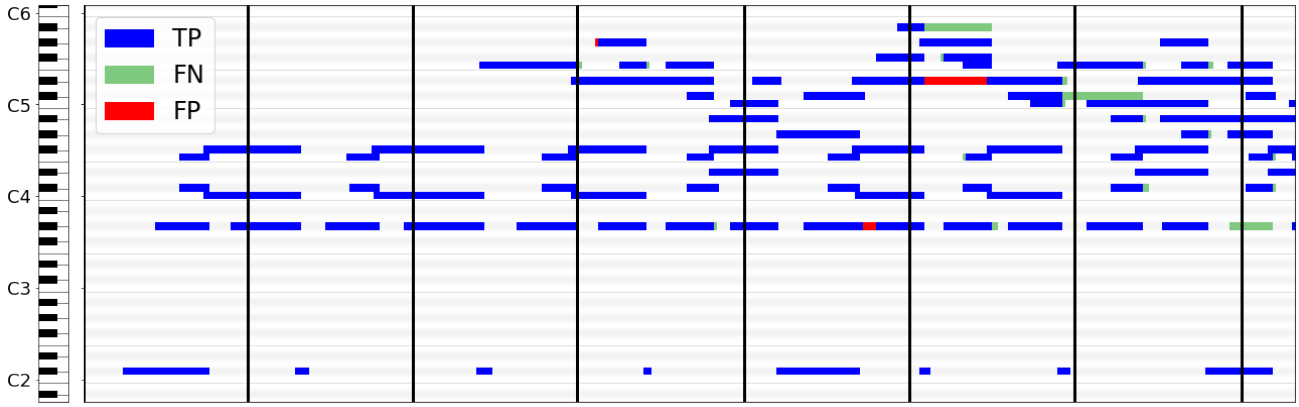



**Figure 2**. Section of a piano roll rendering of model event output on Chopin's *Berceuse Op. 57 in D-flat Major* from the MAESTRO validation set versus the ground truth. Black vertical lines represent segment boundaries during inference. True positive (TP) frames are marked in blue, false negatives (FN) in green, and false positives (FP) in red. Note that the model successfully predicts note-off events for notes where the note-on event happened in a different segment.

the same inputs can be trained to predict only onsets (using just the Note, Time, and EOS events) or onsets, offsets, and velocities (using the full vocabulary above). The only change required is using a different set of tokens as the training target. This is in contrast to previous work where predicting a new feature required adding new output heads (or entire stacks), designing losses for those outputs, and modifying the (often non-differentiable) decoding algorithm to combine all model outputs into the final desired representation.

By using a sequence-to-sequence approach, our model can directly output our desired representation by jointly modeling audio features and language-like output dependencies in a fully differentiable, end-to-end training setting. Adding new output features or changing the task definition is simply a matter of changing the tokens used to describe the target output.

### 3.3 Sequence Length Considerations

Transformers can attend to all tokens in a sequence at every layer, which is particularly suitable to a transcription task that requires fine grained information about pitch and timing for every event. However, this attention mechanism comes at a space complexity of $O(n^2)$ with respect to sequence length $n$. The practical consequence is that most audio sequences used for transcription cannot fit in memory. To get around this problem, we split the audio sequence and its corresponding symbolic description into smaller segments during training and inference.

During training, we use the following procedure for every sequence in a batch:

1. Select a random audio segment from the full sequence for model input. The length of the selected segment can vary from a single input frame to the maximum input length, and the starting position is selected from a uniform random distribution.

2. Select the symbolic segment for the training target that corresponds to the selected audio segment. Because notes may start in one segment and end in another, the model is trained to be able to predict note-off events for cases where the note-on event was not observed.

3. Compute a spectrogram for the selected audio and map the symbolic sequence into our vocabulary (see Section 3.2). Absolute time shifts within the symbolic segment are calculated such that time 0 is the beginning of the segment.

4. Provide the continuous spectrogram input and one-hot-encoded MIDI-like events as a training example for the Transformer architecture.

During inference, the following procedure is used:

1. Split the audio sequence into non-overlapping segments using the maximum input length when possible, and then compute spectrograms.

2. For each segment in turn, provide the spectrogram as input to the Transformer model and decode by greedily selecting the most likely token according to the model output at each step until an EOS token is predicted. Any tokens occurring after a time shift beyond the length of the audio segment will be discarded.

3. Concatenate the decoded events from all segments into a single sequence. After concatenation, there may still be note-off events with no corresponding note-on; we remove these. If we encounter a note-on event for a pitch that is already on, we end the note and start a new one. At the end of the sequence, we end any active notes that are missing note-off events.

The model is surprisingly capable at predicting note-on or note-off events where the corresponding event is in a different segment, as illustrated in Figure 2. This ability is also empirically demonstrated by the model's results on the Onset, Offset, & Velocity F1 scores in Section 4.2.

## 4. EXPERIMENTS

We trained our model with the Adafactor optimizer [35] using a batch size of 256, a constant learning rate of $1e{-}3$, and dropout set to .1 for both sub-layer outputs and embedded inputs. Batch size was selected to maximize training throughput because other batch sizes we tried during initial experimentation did not seem to make a difference in final performance. The learning rate and dropout values were set to the same values used by T5 for fine-tuning tasks.

Input spectrograms were calculated using the Tensorflow [36] *tf.signal* library. We used an audio sample rate of 16,000 kHz, an FFT length of 2048 samples, and a hop width of 128 samples. We scaled the output to 512 mel bins (to match the model's embedding size) and used the log-scaled magnitude.

Input sequences were limited to 512 positions (511 spectrogram frames plus a learnable EOS embedding), and outputs were limited to 1024 positions (1023 symbolic tokens plus a learnable EOS embedding). This corresponds to a maximum segment length of $4.088$ seconds. We used 512 input positions to match the sequence length of T5, but future work could explore if other sequence lengths result in better performance. 1024 output positions were used because we found that 512 output positions were not always sufficient to symbolically describe the input audio.

We trained all models on 32 TPUv3 cores, resulting in a per-core batch size of 8. We used this configuration for training speed, but the model is small enough to train on a single TPUv2 instance (8 cores). Based on validation set results, overfitting did not seem to be a problem, so we allowed training to progress for 400K steps, which took about 2.5 days for our baseline models.

### 4.1 Datasets

To evaluate the performance of our model on the task of piano transcription, we use the MAESTRO dataset [2], which contains about 200 hours of virtuosic piano performances captured with fine alignment between audio and ground truth note annotations. For comparison against previous transcription work, we train on MAESTRO V1.0.0, but for other studies we use MAESTRO V3.0.0 because it contains an additional 92 performances containing 26 hours of data. MAESTRO V3.0.0 also contains *sostenuto* and *una corda* pedal events, though our model (and evaluation) does not make use of these. We also do not model sustain pedal events directly as in Kong et al. [19], instead extending note durations while the sustain pedal is pressed similar to Hawthorne et al. [2].

### 4.2 Evaluation

In evaluating the performance of a piano transcription system, we use the Note F1 score metric: the harmonic mean of precision and recall in detecting individual notes. This involves matching each predicted note with a unique ground truth note based on onset time, pitch, and optionally offset time. Additionally, onset velocity can be used to discard matches with drastically different velocities. We primarily use an F1 score that takes into account onsets, offsets, and velocities. We also include results for F1 scores that consider only onsets or onsets and offsets. We defer to the *mir_eval* [37] library for a precise definition of the (standard) transcription metrics we use.

Because piano is a percussive instrument, it is generally easier (and also more perceptually important) to accurately identify note onsets compared to offsets [38]. We use *mir_eval*'s default match tolerance of 50 ms for onsets and the greater of 50 ms or 20% of the note's duration for offsets.

### 4.3 Comparison to Previous Work

We compare our sequence-to-sequence approach with the reported scores from previous piano transcription papers on V1.0.0 of the MAESTRO dataset in Table 1. Our method is able to achieve competitive F1 scores compared to the best existing approach while being conceptually quite simple, using a generic architecture and decoding algorithm and standard representations.

### 4.4 Ablation Study

We perform an ablation study on some of the components of our model using V3.0.0 of the MAESTRO dataset in Table 1. First, we verify the flexibility of this architecture to describe the input audio using a different set of features. We modify the symbolic data to describe only onsets by using just the Note, Time, and EOS events. The model trains successfully and achieves a high F1 score on this modified onset-only task.

Next, we investigate different input representations. For "STFT", we remove the log mel scaling after FFT calculation. This results in an input frame size of 1025, which is projected to the model embedding size of 512 by the dense layer. For "raw samples", we simply split the audio samples into segments based on the hop width used for the spectrograms (128 samples) and use those directly as input, again projected to the embedding size by the dense layer. Both of these configurations train successfully, but do not perform as well as log mel input. We suspect this is because the mel scaling produces useful features that the model would otherwise have to use some of its capacity to extract.

We also verify that absolute time shifts are a better fit for this architecture by training a model with relative time shifts. As expected, it does not perform as well. Further, we noticed that the note-based evaluation metrics on the validation set varied dramatically during training, with sometimes as much as a 15 point difference in onset F1 score between adjacent validation steps. We hypothesize this is because small changes in relative time shift prediction are magnified when accumulated across a sequence to determine the absolute times needed for the metrics calculation; i.e., relative time shifts cause the resulting transcriptions to drift out of alignment with the audio.

Finally, we investigate if a larger model size would improve performance. We scaled up our model size based on the "base" configuration from T5. Specifically, we

| | Model | Onset, Offset, & Velocity F1 | Onset & Offset F1 | Onset F1 |
|---|---|---|---|---|
| MAESTRO V1.0.0 | Transformer (ours) | **82.18** | **83.46** | 95.95 |
| | Kong et al. 2020 [19] | 80.92 | 82.47 | **96.72** |
| | Kwon et al. 2020 [21] | – | 79.36 | 94.67 |
| | Kim & Bello 2019 [20] | 80.20 | 81.30 | 95.60 |
| | Hawthorne et al. 2019 [2] | 77.54 | 80.50 | 95.32 |
| MAESTRO V3.0.0 | Transformer | **82.75** | **83.94** | 96.01 |
| | Onsets only vocabulary | – | – | **96.13** |
| | STFT input | 81.81 | 82.92 | 95.44 |
| | Raw samples input | 74.79 | 77.26 | 92.35 |
| | Relative time shifts output | 66.25 | 67.35 | 80.02 |
| | "Base" model size (100K steps) | 81.41 | 82.78 | 95.60 |

**Table 1**. MAESTRO test set results. Comparisons against previous work are done using V1.0.0 and comparisons against different model configurations are done using V3.0.0 because of its larger size. All Transformer models were trained to 400K steps except for the "Base" configuration which was trained to 100K steps.

modified the following hyperparameters: $d_{\text{model}} = 768$, $d_{\text{ff}} = 2{,}048$, 12 heads for attention, and 12 layers each in the encoder and decoder. These changes resulted in a model with 213M parameters, as opposed to our "small" configuration which had only 54M. This model quickly overfit the training dataset, with scores on the validation set starting to decline after 100K steps, so we stopped training at that point. Even with early stopping, this model does not perform as well as our "small" configuration, clearly demonstrating that even though piano transcription is a fairly complicated task, it does not require a particularly large Transformer.

## 5. CONCLUSION AND FUTURE WORK

We have shown that a generic Transformer architecture trained to map spectrograms to MIDI-like output events with no pretraining can achieve state-of-the-art performance on automatic piano transcription. We see this as an appeal to simplicity; we used standard formats and architectures as much as possible and were able to achieve results on par with models customized for piano transcription. Possibly the main source of complexity in our setup is the splitting of examples into segments; future work could include the investigation of sparse attention mechanisms to enable the transcription of an entire piece of music in a single encoding and decoding pass. Also worth exploring would be the use of distillation [39] or related techniques to enable models like this to run in realtime on mobile devices or the web.

Our results suggest that a generic sequence-to-sequence framework with Transformers might also be beneficial for other MIR tasks, such as beat tracking, fundamental frequency estimation, chord estimation, etc. The field of Natural Language Processing has seen that a single large language model, such as GPT-3 or T5, has been capable of solving multiple tasks by leveraging the commonalities between tasks. We are excited by the possibility that similar phenomena could be possible with MIR tasks, and we hope that these results point toward possibilities for creating new MIR models by focusing on dataset creation and labeling rather than custom model design.

## 6. REFERENCES

[1] V. Emiya, R. Badeau, and B. David, "Multipitch estimation of piano sounds using a new probabilistic spectral smoothness principle," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 18, no. 6, pp. 1643–1654, 2009.

[2] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the MAESTRO dataset," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=r1lYRjC9F7

[3] C. Hawthorne, E. Elsen, J. Song, A. Roberts, I. Simon, C. Raffel, J. Engel, S. Oore, and D. Eck, "Onsets and Frames: Dual-objective piano transcription," in *ISMIR*, 2018.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeurIPS*, 2017.

[5] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=YicbFdNTTy

[6] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text

Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[7] A. Jaegle, F. Gimeno, A. Brock, A. Zisserman, O. Vinyals, and J. Carreira, "Perceiver: General perception with iterative attention," *arXiv preprint arXiv:2103.03206*, 2021.

[8] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with Transformers," in *ECCV*, 2020.

[9] A. Ramesh, M. Pavlov, G. Goh, S. Gray, C. Voss, A. Radford, M. Chen, and I. Sutskever, "Zero-shot text-to-image generation," *arXiv:2102.12092*, 2021.

[10] K. Lin, L. Wang, and Z. Liu, "End-to-end human pose and mesh reconstruction with Transformers," in *CVPR*, 2021.

[11] L. Dong, S. Xu, and B. Xu, "Speech-Transformer: a no-recurrence sequence-to-sequence model for speech recognition," in *ICASSP*, 2018.

[12] N.-Q. Pham, T.-S. Nguyen, J. Niehues, M. Müller, S. Stüker, and A. Waibel, "Very deep self-attention networks for end-to-end speech recognition," in *Interspeech*, 2019.

[13] N. Li, S. Liu, Y. Liu, S. Zhao, and M. Liu, "Neural speech synthesis with Transformer network," in *AAAI*, 2019.

[14] Y. Gong, Y.-A. Chung, and J. Glass, "AST: Audio spectrogram Transformer," *arXiv:2104.01778*, 2021.

[15] MIDI Manufacturers Association and others, "The complete midi 1.0 detailed specification," *Los Angeles, CA, The MIDI Manufacturers Association*, 1996.

[16] N. Boulanger-Lewandowski, Y. Bengio, and P. Vincent, "Modeling temporal dependencies in high-dimensional sequences: Application to polyphonic music generation and transcription," in *ICML*, 2012.

[17] J. Nam, J. Ngiam, H. Lee, and M. Slaney, "A classification-based polyphonic piano transcription approach using learned feature representations," in *ISMIR*, 2011.

[18] S. Böck and M. Schedl, "Polyphonic piano note transcription with recurrent neural networks," in *ICASSP*, 2012.

[19] Q. Kong, B. Li, X. Song, Y. Wan, and Y. Wang, "High-resolution piano transcription with pedals by regressing onsets and offsets times," *arXiv:2010.01815*, 2020.

[20] J. W. Kim and J. P. Bello, "Adversarial learning for improved onsets and frames music transcription," in *ISMIR*, 2019.

[21] T. Kwon, D. Jeong, and J. Nam, "Polyphonic piano transcription using autoregressive multi-state note model," in *ISMIR*, 2020.

[22] R. Kelz, S. Böck, and G. Widmer, "Deep polyphonic ADSR piano note transcription," in *ICASSP*, 2019.

[23] A. Elowsson, "Polyphonic pitch tracking with deep layered learning," *Journal of the Acoustical Society of America*, vol. 148, no. 1, pp. 446–468, 2020.

[24] J. Engel, R. Swavely, L. H. Hantrakul, A. Roberts, and C. Hawthorne, "Self-supervised pitch detection by inverse audio synthesis," in *ICML Workshop on Self-Supervision in Audio and Speech*, 2020.

[25] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," in *NeurIPS*, 2020.

[26] M. Awiszus, "Automatic music transcription using sequence to sequence learning," Master's thesis, Karlsruhe Institute of Technology, 2019.

[27] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[28] K. Ullrich and E. van der Wel, "Music transcription with convolutional sequence-to-sequence models," in *ISMIR*, 2017.

[29] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin *et al.*, "Tensorflow: Large-scale machine learning on heterogeneous distributed systems," *arXiv preprint arXiv:1603.04467*, 2016.

[30] J. Heek, A. Levskaya, A. Oliver, M. Ritter, B. Rondepierre, A. Steiner, and M. van Zee, "Flax: A neural network library and ecosystem for JAX," 2020. [Online]. Available: http://github.com/google/flax

[31] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/google/jax

[32] S. Oore, I. Simon, S. Dieleman, D. Eck, and K. Simonyan, "This time with feeling: Learning expressive musical performance," *Neural Computing and Applications*, vol. 32, no. 4, pp. 955–967, 2020.

[33] A. Friberg and J. Sundberg, "Perception of just-noticeable time displacement of a tone presented in a metrical sequence at different tempos," *Journal of The Acoustical Society of America*, vol. 94, no. 3, pp. 1859–1859, 1993.

[34] S. Handel, *Listening: An introduction to the perception of auditory events.* MIT Press, 1993.

[35] N. Shazeer and M. Stern, "Adafactor: Adaptive learning rates with sublinear memory cost," in *International Conference on Machine Learning*. PMLR, 2018, pp. 4596–4604.

[36] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng, "TensorFlow: Large-scale machine learning on heterogeneous systems," 2015. [Online]. Available: https://www.tensorflow.org/

[37] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, D. P. Ellis, and C. C. Raffel, "mir_eval: A transparent implementation of common MIR metrics," in *ISMIR*, 2014.

[38] A. Ycart, L. Liu, E. Benetos, and M. Pearce, "Investigating the perceptual validity of evaluation metrics for automatic piano music transcription," *Transactions of the International Society for Music Information Retrieval*, 2020.

[39] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[40] N. Shazeer, "Glu variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.